

4G ANALYSIS PROGRAM DOCUMENTATION

Killian CALLAC

IMT Atlantique Rennes

July 2022

Internship supervised by Xavier LAGRANGE,
professor at IMT Atlantique,
and Julien SAINT-MARTIN, research engineer.



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Contents

Introduction	2
I Global presentation of the software	3
I.1 Remind of 4G network concepts and perimeter of the software	3
I.2 Software functionalities	4
Initial data processing functionalities	5
Data visualization	6
I.3 Python program structure	8
II Software data formats and processings	9
II.1 Accuver Open Format presentation	9
II.2 "Customized AOF" CSV output format definition	9
Syntax of the file format	9
II.3 Extraction of data from AOF main input file	11
Data to extract	11
AOF file parsing and data extraction	12
Production of PCAP file	13
AOF file data processing.	14
Estimation of geolocations	15
Content of the output file.	16
II.4 Extraction of data from CSV Cartoradio files	16
Cartoradio data processing	16
Cartoradio processing output files	17
II.5 Base station and measurements association process	18
Input data of the assoiation process	18
Geometric calculation on cells	18
Use of vectorized function for the calculation	21
Cell association calculation	22
Warning	25
Content of the association output file	25
III Visualization	26
III.1 JavaScript application structure	26
III.2 Aspects of the inner working of the application	27
Selection of EARFCNs and PCIs	27
Management of layers	27
Automatic generation of the color gradient for heatmaps	27
Updating policy of the UI	28

Introduction

At IMT Atlantique, the Adopnet team uses since 2019 a 4G network analysis software. This software, written with Python programming language, has for main purpose to visualize on a map measurements taken on the 4G mobile network. Usually, these measurements are done in exterior, with one or several devices connected to a measurement tool. Originally, this tool was the ZK-Samp tool, consisting in a measurement device which was taking measurements from one or several phones connected to it. The tool was taking global signal measurements, geolocated and associated to a timestamp. The original program has been developed in order to allow the user to visualize easily global signal measurements in a more concise way, to easily link these measurements to a particular network cell [1].

However, the ZK-Samp tool of the Adopnet team doesn't work anymore, and the company which was producing it no longer exists. The Adopnet team uses now Accuver Xcal-M, a software which can do geolocated global measurements over the network like the ZK-Samp tool, but can also do measurements for each detected cells as well, no matter if the user equipment is actively connected to these cells. Xcal-M produces a data format (Accuver Open Format, or AOF) which is slightly different of the data format produced by the ZK-Samp tool, and it is necessary to adapt the original analysis program to this new format. The format of some intermediate files needs also to be changed for readability reasons, and the visualization must also be improved due to the fact the file format of Xcal-M is more rich.

The purpose of our work on this program is to implement previously mentioned changes ; this report aims to document these changes, for usability, understanding and maintainability purposes. We will first focus on a presentation of the software and how use it, before we present the data processing methods used in the software, to present next the visualization part of the program.

I. Global presentation of the software

In this part, we will first remind the overall structure of a 4G cellular network, the perimeter of the software and the related concepts. We will then present the program itself from an user perspective, by detailing its main functions.

Remind of 4G network concepts and perimeter of the software

Cellular networks are composed of three main parts: the user terminal, commonly called User Equipement (UE) ; an access network (commonly called RAN, for Radio Access Network), which allows the User Equipement to communicate by radio access to a core network ; and the core network, which manage numerous inner functions of the network, as the registering of users, the billing policy, call controls and the connection to exteriors networks (e.g. Internet network) [2].

4G networks are based on the Long Term Evolution (LTE) norm, which uses as access network the Evolved Universal Terrestrial Radio Access Network (e-UTRAN), and as core network the Evolved Packet Core (EPC) [2].

The program has as perimeter the User Equipement and the e-UTRAN network: the User Equipement is used here to do measurements, and these measurements are done on the e-UTRAN network.

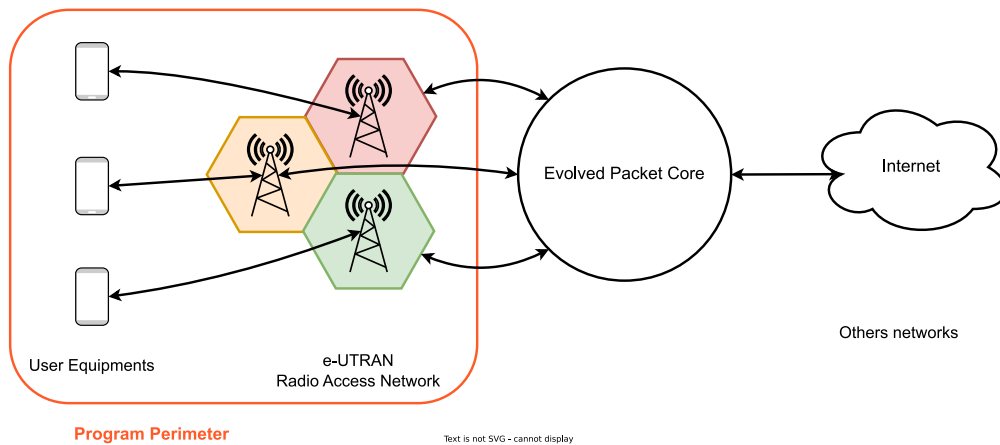


FIGURE 1 – LTE network architecture

A Radio Access Network uses base stations to communicate with User Equipments. These base stations are equipped of several antennas, with can send data to and receive data from User Equipements [2]. Every base station covers a geographical zone called a cell. Antennas of base stations are oriented to cover a specific zone of the cell, called a sector. Usually, cells are divided in three sectors, every sectors often covering an angle of approximately 120° around the base station ; however, it is possible to encounter cells with more or less sectors.

In the e-UTRAN network, bases stations are called eNodeB.

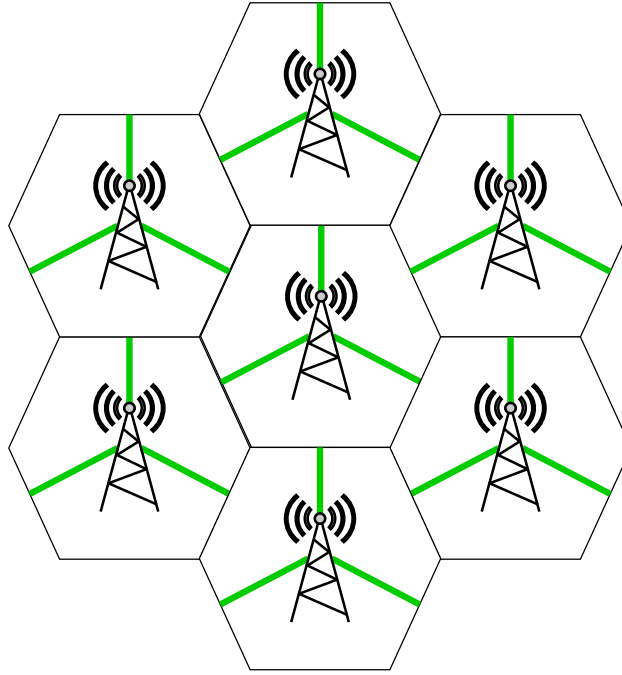


FIGURE 2 – e-UTRAN cells spatial organization. Each hexagonal cell is covered by an eNodeB ; in case of trisectorization, cells are divided in three parts, delimited by green lines on the figure.

eNodeB can emit data on several frequency bands. These frequency bands are designated by a code called EARFCN (Extended Absolute Radio Frequency Channel Number). Data sent from the UE to the network (Uplink, or UL) and data sent from the network to the UE (Downlink, or DL), use different frequency bands.

We will present more detailed aspects of the LTE network in next parts.

Software functionalities

The tool is usable through a graphical user interface which allows the user to access main functionalities. Textual traces can also be found on a command line interface.

Initial data processing functionalities

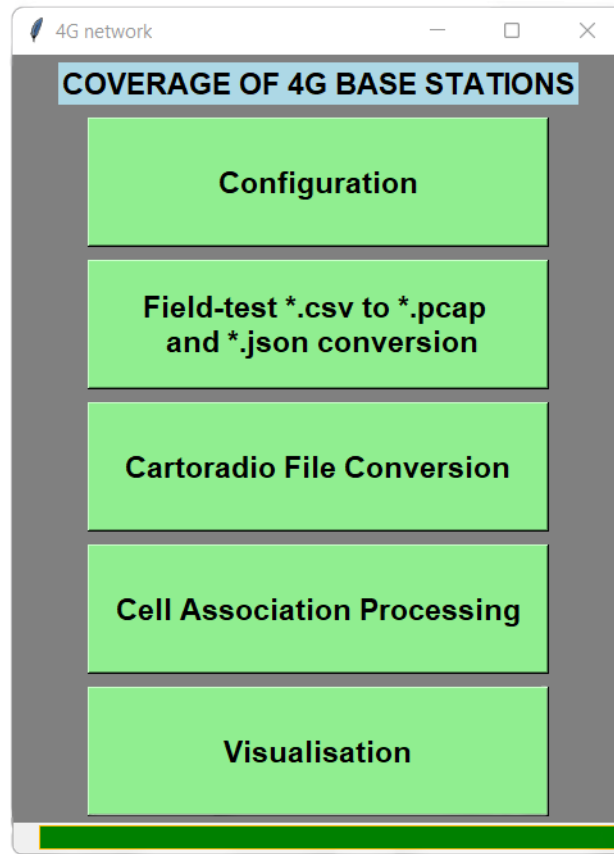


FIGURE 3 – Software user interface.

The user must first click on the "Configuration" button and choose in a dialog a directory: in this directory will be produced output files from the program.

Next, the user can use other functionalities of the program. The **Field-test *.csv to *.pcap and AOF-like *.csv conversion** button allow the user to choose an Accuver Open Format measurement file, and produce a PCAP trace readable by Wireshark, and a CSV¹ file constructed like an AOF file, which contains information about cells, geolocations of measurement points, and measurements classed by EARFCN and cell identifiers. Every measurement in this output file possess a timestamp, which locates information in time.

The user can also use the button **Cartoradio File Conversion** to produce information about antennas in several AOF-shaped CSV output files, from datasets from the ANFR² website Cartoradio. Exportable datasets from Cartoradio contains several CSV files. Two of them are used here: the `Sites_Cartoradio.csv` and `Antennes_Emetteurs_Bandes_Cartoradio.csv`: the first file contains information about base stations, their geolocation, their height and several information about the place where they are located, and the second file contains data about each antenna found on the exported zone, such as directivity information (azimuth), used technology (LTE for 4G), frequency bands or operators. Two output files are produced per operator, a "sites" (eg. `sites_SFR.csv`) file which associates each antenna to its base station data and to a sector delimitation. Sector delimitation data are also produced separately in the file for the visualization. "Rejected" files

¹Commas Separated Values, textual format readable by spreadsheets which represents data tabularly.

²Agence Nationale des Fréquences, french organization that manages the allocation of frequency bands of the country.

(eg. `rejected_SFR.csv` contain same kind of information, except antennas contained in this file are below the ground level: separate these antennas from others allow the software to not visualize underground base station, which can be encountered for example in metro tunnels and stations, in order to avoid to overestimate the cell range, which is in reality very limited.

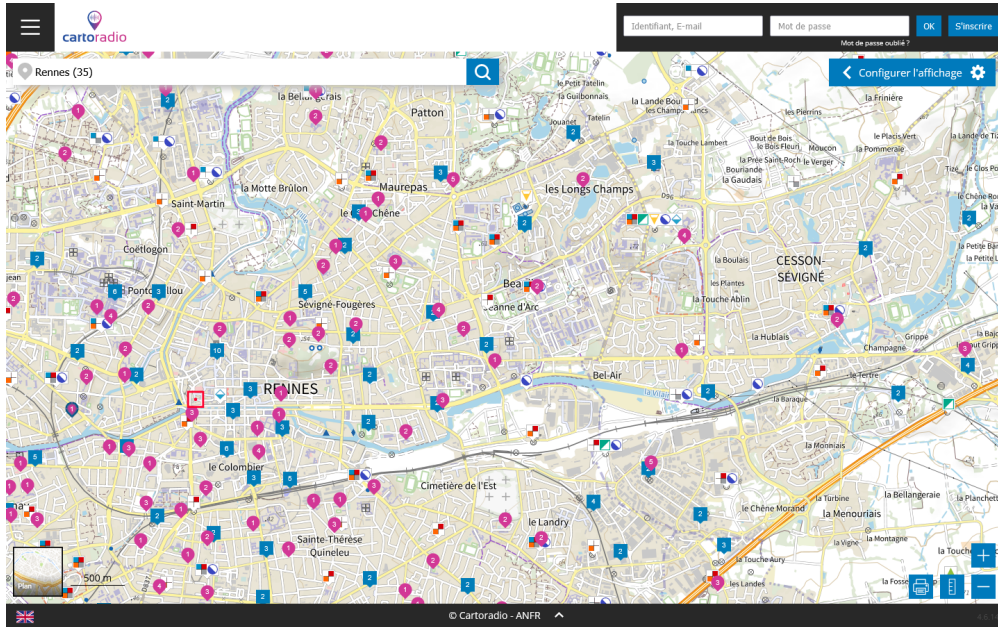


FIGURE 4 – Cartoradio user interface.

The **Cell Association Processing** feature take as input a "sites" file produced by the Cartoradio File Conversion step and a CSV file produced by the Field-test functionality. This feature associates antennas from the first file with measurement points from the second file: details of the association calculation will be given in nexts parts. Groups of points, corresponding to a given cell and a given EARFCN, are also created. Measurement points associated with antennas and measurement values, measurement values and sectors data used by the visualization are stored in a final output CSV file.

Data visualization

The software allows the user to visualize measurements points on a graphical interface. This interface is in fact a local web app which can be used through a web browser.

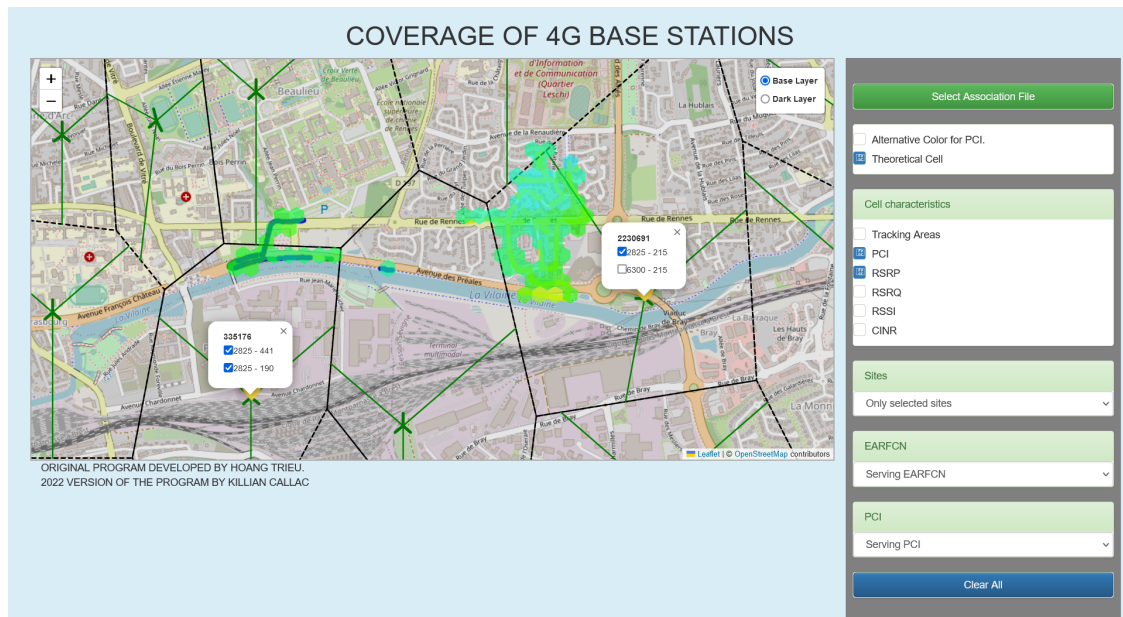


FIGURE 5 – Measurement visualization interface.

The web interface includes a map, which is used to display measurement, and several lateral panels, used to set visualization preferences. The user can load an association CSV file and can display data contained in the file with the "Select Association File" button. This action will have as an effect to display green "crosses", which corresponds to loaded base station: branches of these "crosses" indicate in fact the directivity of each antennas of base stations. Base stations with a yellow mark on them are base stations associated to measurement points; numbers of these associated base stations can be seen by clicking on yellow marks.

The user can use the "Theoretical Cells" checkbox at the left of the screen to show the Voronoi diagram which estimates the delimitation of the cells. From each base station, several thin green lines that extends from the base station to the limits of the cell can be seen: these lines indicate the limits of sectors associated with antennas.

Still at the left of the screen, a group called "Measurements" can be seen. This groups contains several checkboxes. The first checkbox, "Tracking Area", displays colored measurement points on the map. Each color corresponds to an unique Tracking Area. The "PCI" checkbox displays these points, but with each color corresponding to a Physical Cell Identifier, which identify uniquely each cell. An alternative color choice can be displayed by checking "Alternative Color for PCI". Checkboxes RSRP, RSSI, RSRQ and CINR displays different signal measurements around geolocated points, using an hexagonal colored tiling to represent the intensity of these measurement. The meaning of RSRP, RSRQ, RSSI and CINR will be explained later.

In the "Sites" menu, it is possible to select two options: "All sites" display all measurements from all sites, "Only selected sites" allows the user to chose which EARFCN / PCI from an associated it wants to see, by clicking on the associated site pins and check the corresponding EARFCN / PCI.

In the bottom right corner of the screen, frames "EARFCN" and "PCI" contains two drop-down lists, which respectively allow the user to choose an EARFCN and a PCI. If an EARFCN and / or PCI is chosen, measurements will be filtered on the map, allowing to only see global measurements with corresponding EARFCN / PCI. Other possible choices are serving EARFCNs and PCIs, which displays EARFCNs and / or PCIs from the serving antennas, and "All EARFCNs" / "All PCIs" options allowed the user to visualise global measurements for all EARFCNs and PCIs.

Python program structure

The Python program is divided into several modules, several of them being entirely dedicated to one of the functionalities of the software.

The bootable part of the program is situated in the module `Interface.py`, which contains the GUI class, which is the main frame (window) of the program. The library used for the interface is `tkinter`, which is distributed in Python standard library. This module must be run in order to execute the program.

The module `xcalyzer.py` is dedicated to the "Field-test *.csv to *.pcap and AOF-like *.csv conversion" functionality. It contains the parsing class `XcalConverter` for the AOF format, which arranges data stored in the input AOF file to produce the output file of this functionality.

The module `cartoradio.py` contains functions to read Cartoradio "sites" and "antennas" files, and produce output files containing information about antennas for each operator.

The module `association.py` contains the class `CellAssociator`, which is used to associate antennas data and measurements.

There is also several utility modules: `csvtools.py`, which contains class to read custom AOF-like CSV format, which will be described later, `dictutils.py`, which is a module that contains utility functions to manage data dictionaries, and `pcaputils.py`, used to generate easily data in PCAP files. Legacy module `constantPath.py` is also used as it provides useful paths functions.

Several libraries are also used in the program. These libraries are the following:

- Pandas: used to do data treatments (grouping, natural joining, filtering) on massive datasets.
- SciPy: used here to calculate theoretical cells.
- Shapely: used to geometrical calculations.
- Numpy: used here to do vectorized calculations.

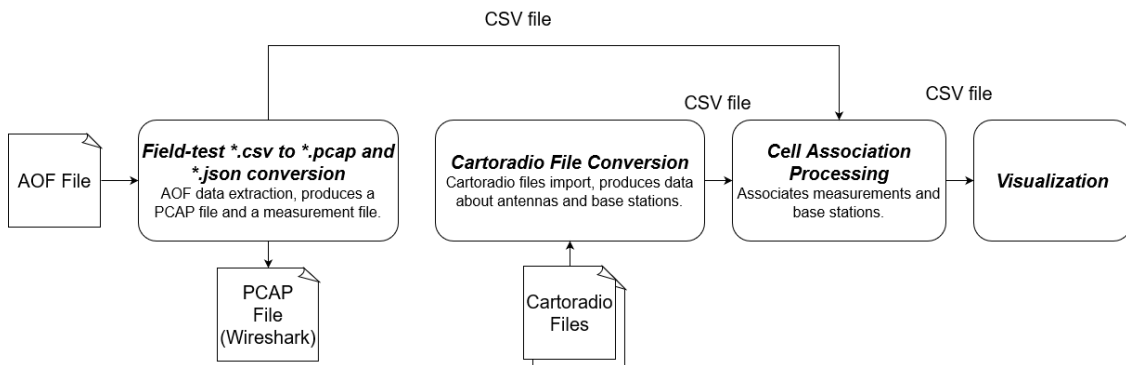


FIGURE 6 – Software architecture.

II. Software data formats and processings

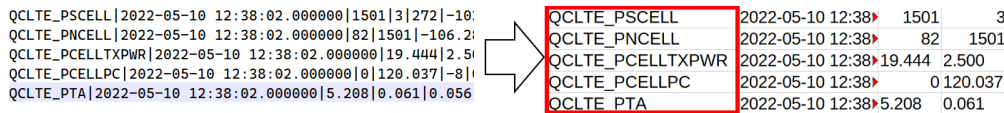
In this section, we will present data formats used as inputs and outputs of the software, and related data processings. We will first present the Accuver Open Format (AOF), used as main input format, then we will define the common format used in each functionality of the software, before we present data treatments on these formats.

Accuver Open Format presentation

Accuver Xcal uses as output format the "Accuver Open Format" (AOF), a textual format which contains measurement traces following a textual, tabular structure. This format is in fact an extension of the Comma Separated Values (CSV) format, commonly supported by most of spreadsheets softwares. Basically, the CSV format arranges data tabularly, each line of the file corresponding to a row. Columns are separated by a separator character (usually the ',' character, but others characters can be used in other languages, e.g. ';' in French). The AOF format uses as separator character '|'. Each row, and so each line, corresponds to a data trace, with fields separated by the separator character.

The most notable particularity of the AOF format is the fact the file is splitted in three parts, delimited by two delimiters which take each one line. The first part, the **AOF_Information** part (delimited by <AOF_Information Start> and <AOF_Information End>, contains data about Accuver Xcal, such as the version number. The second part, the **Description** part (delimited by <Description Start> and <Description End>), defines each trace type, and its fields names and types. A trace definition is divided in two lines, each line beginning by the trace name: the first line defines the name of each field, and the second line defines types of these fields. The third part, the **Content** part (delimited by <Content Start> and <Content End>), contains data itself, with each trace identified by its name, at the beginning of the line.

A great advantage of the AOF format is, because it is based on the CSV format, it can be read on a spreadsheet software. A filter can be used to visualize each trace type, with corresponding headers and data types visible at the top of the table.



QCLTE_PSCCELL	2022-05-10 12:38:02.000000	1501	3	272	-10
QCLTE_PNCELL	2022-05-10 12:38:02.000000	82	1501	-106	2
QCLTE_PCELLTXPWR	2022-05-10 12:38:02.000000	19.444	2.500		
QCLTE_PCELLPC	2022-05-10 12:38:02.000000	0	120.037	-8	
QCLTE_PTA	2022-05-10 12:38:02.000000	5.208	0.061	0.056	

FIGURE 7 – AOF file, visualized on a text editor and on a spreadsheet. Figure by Killian CALLAC, extracted from [3]

"Customized AOF" CSV output format definition

Each data processing step of the software produces different output files. These files use a specific data format. We based this format on the AOF file format, to replace JSON³ output files previously used, which were not easily readable by a human. As we previously mentioned it, the AOF file format is more easier to read, especially when displayed by a spreadsheet, a quality which is appreciated for an output file format.

Syntax of the file format

Formally, the syntax of the format is, using Python regular expressions⁴:

³JavaScript Object Notation

⁴<https://docs.python.org/3/library/re.html#regular-expression-syntax>

```

DEFINE\n
(\w+\|\\w+(\|\\w+)*\n)*
CONTENT\n
(\w+\|([\^\\s|]|_)*(\|([\^\\s|]|_)*))*\n)*

```

FIGURE 8 – Syntax of the AOF / CSV file format.

The file is divided in two parts, a **DEFINE** part and a **CONTENT** part. In the **DEFINE** part, data entry types are specified: at the beginning of the line, a non-empty string, using only alphanumeric characters, defines the name of the entry. Following fields names, separated by the character '|', should be defined with a non-empty, alphanumeric string; each entry definition, which take one line, should contain at least an entry name and one field name. There can be zero, one ore more entry definitions. The **CONTENT** part contains data entries, which occupy one line each. Data entries are identified by entries names (non-empty alphanumeric strings), and several values, which can be empty, and made of all characters which are not spacing characters, unless the '|' character, which is used to separate values of an entry.

More informally, an example of valid CSV / AOF file can be found below:

```

1 DEFINE
2 entryName1|fieldName1|fieldName2|fieldName3
3 entryName2|fieldName4|fieldName5|fieldName6|fieldName7|fieldName8
4 entryName3|fieldName9|fieldName10
5 CONTENT
6 entryName1|10|test|50
7 entryName2|never|gonna|give|you|up
8 entryName1|something|test|
9 entryName3|3.14|42
10 entryName3||
11 entryName1||1|10|
12 entryName2|never|gonna|let|you|down

```

FIGURE 9 – Example of a valid AOF/CSV text. Note there is no need to have same length entries, or non empty values in entries.

In the program, this file format is handled by the module `csvtools`. This module contains a writer class, `CSVWriter`, and a reader class, `CSVReader`. `CSVWriter` requires to write in the file an header, which is a dictionary used to define the **DEFINE** part, which use as keys names of entries and as values lists of field names . A line can be added to the **CONTENT** file with a list beginning with the corresponding entry name, followed by values, associated by indexes to corresponding fields in the header.

`CSVReader` reads file data. The file is firstly opened temporarily, the corresponding header is generated first following the **DEFINE** part, and syntactic and semantic controls are done on the file. Secondly, the file is closed and reopened, and the user can read line per line the **CONTENT** section.

As the language of the file does not define nested structures, and follows a linear sequence, we can identify it as a regular language, which can be recognized by a finite state machine. This finite state machine use as symbols first elements of each lines: at each transition, a new line is read. For the first reading of the file, we implemented the following finite state machine, using actions (numbered in **bold** on transitions) to do syntactic and semantic controls:



FIGURE 10 – AOF-like CSV recognition finite state machine. "*" symbols mean here every possible symbol can be accepted; note that a valid symbol on the FSM can be refused by the action associated to the transition.

Corresponding actions are:

- (1): Use the first symbol of the line as entry name. Verify that the line contains field names. Verify that entry name follows the pattern $\backslash w^+$. For each field name, verify that the file name follows the pattern $\backslash w^+$. Expand header dictionary.
- (2): Use the first symbol of the line as entry name. Verify that follows the pattern $\backslash w^+$ and belongs to header keys. Verify that the line contains values. For each value, verify that values follows the pattern $([\sim \backslash s] | \lfloor \rfloor)$.

Extraction of data from AOF main input file

The input file which contains measurements is the AOF file produced by Accuver Xcal. Its analysis is focused on basic syntactic controls, used to locate data, and the extraction of these data.

Data to extract

The main purpose of the software is to visualize 4G network coverage using signal measurements. In particular, the program uses specific signals called "Reference Signals" (RS). These Reference Signals are special signals already known by the User Equipment, which are used to estimate channels to use or to do measurements on the network [2]. Measurements displayed by the software are based on these reference signals. Used measurements are:

- The Reference Signals Received Power (RSRP): average received power of Reference Signals, expressed in dBm⁵. [2], [5].
- The Received Signal Strength Indicator (RSSI): total power measured on the whole frequency band, expressed in dBm [5].
- The Reference Signals Received Quality (RSRQ): quality of Reference Signals, which the ratio between RSRP and RSSI [2], [5].
- The Carrier to Interference and Noise Ration (CINR): ratio between signal power and noise and interference power.

The program also identify each cell using information such as the TAC (Tracking Area Code), the CID (Global Cell Identifier), the PCI (Physical Cell Identifier), and the EARFCN (Extended Absolute Radio Frequency Channel Number) on downlink, which identifies the used frequency band. We can considerate there is one unique couple (EARFCN, PCI) in a large area, which will probably not be entirely covered by the measurement campaign. The program also use PLMN (Public Land Mobile Network) code, which is made of a MCC (Mobile Country Code) number and a MNC (Mobile Network Code) number.

Measurement should also be geolocated.

Data to extract are located in the **Content** part of the AOF file, in the following traces:

⁵decibels-milliwatts, unit of measurement which expresses signal power on a logarithmic scale with a reference of 1 mW (1 mW of power = 0 dBm) [4]

- **QCLTE_CELLINFO**: contains TAC, EARFCN, PCI, PLMN of the serving cell.
- **QCLTE_PSCCELL**: contains information and measurements about the serving cell, such as EARFCN, PCI, RSRP, RSSI, RSRQ and CINR.
- **QCLTE_PNCELL**: same as **QCLTE_PSCCELL**, but contains data about every neighbour cell and without CINR.

Each trace possess a timestamp, which identify the moment of the measurement. Geolocation data can also be found in **GPS** traces.

As the AOF data processing produces also PCAP files readable by Wireshark, network messages themselves exchanged between the UE and the network must be extracted. More precisely, these messages are RRC (Radio Resource Control) messages, which are used for many purposes: main uses of these messages are the allocation of resources on the network, signaling, the sending of information about the network or measurements on the network... [2] These RRC messages can be found in the **QCLTE_RRCMSG_V2** traces of the AOF file, in an hexadecimal form.

AOF file parsing and data extraction

The class used in the program to parse the AOF file is **XcalConverter**, in the **xcalyzer** module. The file parsing is done in two step: a first step of basic syntactic control and file data "scanning", and a second step of where the input file is read a second time, and output data are produced in an output file.

Output file data must be organized in a specific form. Input file measurements are organized like this, with RSRP as example of measurement:

Timestamp	EARFCN1	PCI1	RSRP1	EARFCN2	PCI2	RSRP2	...
t_0	3000	21	-94	151	21	-104	...
t_1	151	21	-87	3000	21	-97	...
t_2	151	21	-82	3000	21	-92	...
t_3	3000	21	-89	3000	14	-96	...

FIGURE 11 – Input file data organization.

We can see measurements are organized by timestamp, with each row containing measurements for different EARFCN and PCI. Note that rows may not have measurements for each EARFCN and PCI, and EARFCN / PCI can appears in a different order between two rows. As the software allows the user to choose measurements with specific EARFCN and PCI, it reorganizes data in the output file as follows:

Timestamp	EARFCN	151	3000	3000	...
	PCI	21	14	21	...
t_0	RSRP	-104		-94	...
t_1	RSRP	-87		-97	...
t_2	RSRP	-82		-92	...
t_3	RSRP		-96	-89	...

FIGURE 12 – Output file data organization.

In the output file, each row is always associated to a timestamp, but columns, using the first two rows, are now associated to unique EARFCN / PCI couples. Several cells are empty, because there is EARFCN / PCI which are not associated to a RSRP on several timestamps. This representation

of measurements can also be applied on RSRQ and RSSI.

To produce these files, it is needed to know each EARFCN / PCI couple: these couples are identified during syntactic control step. This step ensures that file sections are well formed (correct definition of delimiters). This analysis is done by the `XcalConverter.first_read` function which implements the following finite state machine with actions, which uses as input symbols first element of each lines:

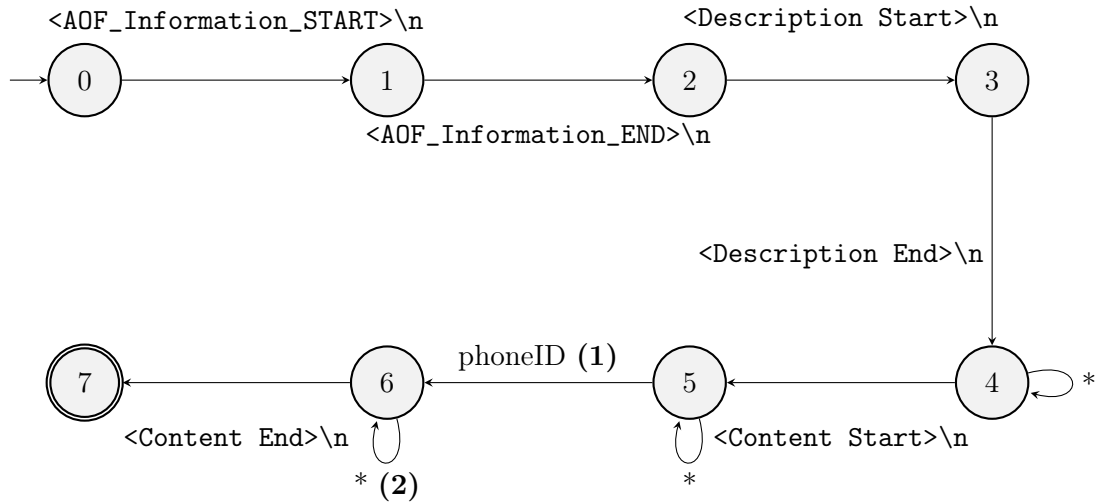


FIGURE 13 – AOF input file analysis finite state machine. "*" character represents every possible symbol, bold numbers are action numbers.

Finite state machine actions are:

- (1): Get the phone ID, to use it in the name of the output file.
- (2): If `QCLTE_RRCMSG_V2` found, write the message in the corresponding intermediate file ; if `QCLTE_PSCELL` or `QCLTE_PNCELL` found, populate EARFCN / PCI list from measurements taken ; if `QCLTE_CELLINFO` found, set MCC and MNC values if they are not set.

Production of PCAP file

During this first step of file analysis, the PCAP file is produced using several tools. These tools are `text2pcap`, which write PCAP files using text file input, `mergpcap`, which can merge several PCAP files into one, and `reordercap`, which can reorder chronologically packets in a PCAP file.

As we mentioned it, the PCAP file contains RRC messages. These message are dispatched on several logical channels (such as BCCH, PCCH...) and transport channels (BCH, DL-SCH...), used to distinguish types of messages [2]. Wireshark uses *dissectors* to decode traces, which are parsers that decode network packet. For each channel, there is a dedicated dissector. The strategy is to produce temporary text files for each dissectors, which contains messages dates and traces in a text format in dedicated files, then to convert them into PCAP files using `text2pcap`.

```

1 2022-05-10 12:39:50.564365
2 0000 60 48 20 21 C0 FA 97 71 90 48 17 50 88 40 42 02 10 A0 88 84 6C
   C0
3
4 2022-05-10 12:39:50.564472
5 0000 00 02 59 3B 18 5F E5 42 D0 36 05 9A 80 D8 00 08 02 44 4E 31 D6 8
   A B0 7A 00 19 00 18 04 7B 31 A6 16 D0 61 60 08 03 10 00 00
6
7 2022-05-10 12:39:50.564594
8 0000 00 05 22 44 FF E2 6D 17 5E C4 A0

```

FIGURE 14 – Extract from a file used by `text2pcap` generated from BCCH channel. Messages are dated and represented in hexadecimal notation

All of generated PCAP files are then merged into one PCAP file with `mergcap`, then this new PCAP file is reordered with `reordercap`.

AOF file data processing.

After the first step of parsing and producing of PCAP files, the AOF file is read a second time to extract data from `QCLTE_PSCCELL`, `QCLTE_PNCELL` and `QCLTE_CELLINFO`. This second read is done by the `XcalConverter.second_read` function.

For this processing, data are structured in a Pandas DataFrame⁶ like dictionary, which represents tabularly data. Given a set of fields `{f1, f2, f3, ...}` and rows `(v11, v12, v13, ...)` and `((v21, v22, v23, ...))`, the corresponding dictionary is:

```

1 {
2     f1: [v11, v21],
3     f2: [v12, v22],
4     f3: [v13, v23],
5     ...
6 }

```

FIGURE 15 – Python "DataFrame" dictionary.

```

1 {
2     f1: [v11, v21],
3     f2: [v12, v22],
4     f3: [v13, v23],
5     ...
6 }

```

FIGURE 16 – Python "DataFrame" dictionary.

If we consider the example of 12, the structure is modified as shown in fig 17.

⁶<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

```

1 {
2     "EARFCN1": [3000, 151, 151, 3000],
3     "PCI1": [21, 21, 21, 21],
4     ...
5 }

```

FIGURE 17 – Example of Python "DataFrame" dictionary.

In this dictionary, the row i corresponds to the index i of lists associated to fields. The advantage of this data structure compared to Pandas Dataframe is it is more time efficient to modify.

In particular, to store measurements by columns each dedicated to an EARFCN / PCI using a this dictionary data structure, we use EARFCNs and PCIs found during the first reading of the file. Measurement from QCLTE_PSCCELL and QCLTE_PNCELL are stored following this structure, each row being associated to a unique timestamp. In the case where several measurements with the same timestamp are found, values located in columns of the first measurement row are always kept, and values located in columns of the others measurement are copied in the first measurement row only if the corresponding column in the first measurement is empty. The name of corresponding entries in the output file is MEASUREMENT.

Concerning QCLTE_PSCCELL measurement values, they are also stored with the CINR in a SERVING_MEAS entry, which corresponds to the serving cell. QCLTE_CELLINFO data are stored in CELLINFO entries.

Estimation of geolocations

Measurements are geolocated in the AOF file with GPS traces. Since there is no GPS traces for each measurement, the location of measurement point is estimated by linear interpolation.

Let $\vec{G}_1, \vec{G}_2 \in [-90, 90] \times [-180, 180]$ two vectors which represent two geolocations respectively taken at timestamps t_1 and t_2 , $t_1 \leq t_2$ and $t_2, t_2 \in \mathbb{R}$, with:

$$\vec{G}_1 = \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} \text{ and } \vec{G}_2 = \begin{pmatrix} x_2 \\ y_2 \end{pmatrix} \quad (1)$$

x_1, x_2 being latitudes and y_1, y_2 longitudes, and M the sets of measurements taken between the two geolocations. Let $t : M \rightarrow [t_1, t_2]$ the function which associates every measurements of M to their timestamp between t_1 and t_2 . Geolocations of measurements in M is estimated with the following function $\hat{G}$:

$$\hat{G} : M \longrightarrow [x_1, x_2] \times [y_1, y_2] \quad (2)$$

$$m \longmapsto \begin{cases} \left(\frac{t(m) - t_1}{t_2 - t_1} \times (x_b - x_a) + x_a \right) \\ \left(\frac{t(m) - t_1}{t_2 - t_1} \times (y_b - y_a) + y_a \right) \end{cases} \text{ if } t_1 \neq t_2 \quad (3)$$

$$\vec{G}_1 \text{ otherwise}$$

It is possible that a series of measurement does not begins or does not terminates with a geolocation. In this case, we consider that $\vec{G}_1 = \vec{G}_2$.

Content of the output file.

The output file contains the following fields:

- **MEAS_EARFCNS**: list of all EARFCNs found, by ascending order. These EARFCNs are associated to PCIs in **MEAS_EARFCNS** line. The **MEAS_EARFCNS** line is used as a header line for the table of measurements.
- **MEAS_PCIS**: list of all PCIs found, ordered by EARFCNs (**MEAS_EARFCNS**). For a same EARFCN, corresponding PCIs are classed by ascending order.
- **CELLINFO**: contains information about a cell detected by the UE, such as EARFCN, PCI, TAC, CID and PLMN of this cell.
- **MEASURE_SERVING**: contains information about the current serving cell (EARFCN, PCI) and measurement on Reference Signals (RSRP, RSSI, RSRQ, CINR).
- **MEASUREMENT**: stores measurements on reference signals of serving and neighbour cells, with each column associated to an EARFCN and PCI using **MEAS_EARFCNS** and **MEAS_PCIS**.

Extraction of data from CSV Cartoradio files

This functionality of the software uses open data from Cartoradio, the ANFR website which reports radio antennas on French territory. These data are exportable in classic CSV format. The functionality produces several CSV file, one by operator, following the format used in previous parts, which contains informations about antennas of the operator which corresponds to the file and antennas sectors data.

Files taken in input are **Sites_Cartoradio.csv** and **Antennes_Emetteurs_Bandes_Cartoradio.csv**. These files contains the following data:

- **Sites_Cartoradio.csv**: base station number, geolocation, postal address, city, support type...
- **Antennes_Emetteurs_Bandes_Cartoradio.csv**: support and administrative numbers, antenna number, operator information, antenna type, directivity, technology (eg. LTE, GSM, UMTS...), frequency band...

Cartoradio data processing

We will now designate in this part the file **Sites_Cartoradio.csv** as the "site" file, and **Antennes_Emetteurs_Bandes_Cartoradio.csv** as the "antennas" file. The processing is done by the function `cartoradio.process_cartoradio`.

The processing is based on the old program. [1] CSV datasets are first loaded in Pandas dataframes, using `pandas.read_csv`. Antennas and sites datasets are joined over the support number, then, as it is required to create an output file for each operator, the dataset is grouped by operator (**Exploitant**), and only directive LTE antennas are kept. For each operator group, underground antennas (located in metros, for example) are rejected, and only antennas of base stations with a height superior or equals to 0 are kept. Antennas which doesn't meet this criteria will be stored by in a "rejected" file which corresponds to an operator (eg. **rejected_SFR.csv**). Before writing the current antenna in an output file, the size of its sectors is calculated. Sector calculation is done following the number of antennas found on the base stations.

Let $i, j \in \mathbb{N}$, where i identifies the current base station, and j the current antenna. Let $\alpha_{i,j}$ the azimuth of the antenna j of the base station i . In general, we have $\alpha_{i,j} < \alpha_{i,j+1}$. We designate as L_i the number of antennas on a base station ; we have therefore $j \in [0, L_i - 1]$. Sectors are

delimited with a minimum azimuth $\alpha_{i,j}^{\min}$ and a maximum azimuth $\alpha_{i,j}^{\max}$, expressed in degrees, and are calculated following these formulas [1]:

$$\alpha_{i,j}^{\min} = \begin{cases} \alpha_{i,j} - 60 & \text{if } L_i = 1 \\ \alpha_{i,j-1}^{\max} & \text{if } L_i > 1 \wedge j = L_i - 1 \\ \min \left(\frac{\alpha_{i,j} + \alpha_{i,L_i-1} - 360}{2}, \frac{\alpha_{i,j} - \alpha_{i,j+1}}{2} \right) & \text{if } L_i > 1 \wedge j = 0 \\ \min \left(\frac{\alpha_{i,j} + \alpha_{i,j-1}}{2}, \frac{\alpha_{i,j} + \alpha_{i,j+1}}{2} \right) & \text{if } L_i > 1 \wedge j > 0 \end{cases} \quad (4)$$

$$\alpha_{i,j}^{\max} = \begin{cases} \alpha_{i,j} + 60 & \text{if } L_i = 1 \\ \alpha_{i,0}^{\min} + 360 & \text{if } L_i > 1 \wedge j = L_i - 1 \\ \max \left(\frac{\alpha_{i,j} + \alpha_{i,L_i-1} - 360}{2}, \frac{\alpha_{i,j} - \alpha_{i,j+1}}{2} \right) & \text{if } L_i > 1 \wedge j = 0 \\ \max \left(\frac{\alpha_{i,j} + \alpha_{i,j-1}}{2}, \frac{\alpha_{i,j} + \alpha_{i,j+1}}{2} \right) & \text{if } L_i > 1 \wedge j > 0 \end{cases} \quad (5)$$

Note that unidirectional antennas are supposed to have sector angle of 120°: this hypothesis is based on the fact antennas sectors has often an angle of approximately 120°.

From these calculations, delimiters lines are calculated: these lines are used by the visualization part. They are defined by the geographic coordinates of the base station i : $x_i^{\text{sta}} \in [-90, 90]$ (latitude) and $y_i^{\text{sta}} \in [-180, 180]$ (longitude), and coordinates $x_{i,k}^{\text{del}} \in [-90, 90]$ and $y_{i,k}^{\text{del}} \in [-180, 180]$, based on each calculated azimuth limits $\alpha_{i,k}^{\text{lim}}$ of base station i , where $k \in \mathbb{N}$ is the k -th azimuth limit (note that an azimuth limit can be a minimum azimuth or a maximum azimuth of a sector):

$$x_{i,k}^{\text{del}} = x_i^{\text{sta}} + 0.3 \cos \left(\alpha_{i,k}^{\text{lim}} \times \frac{\pi}{180} \right) \quad (6)$$

$$y_{i,k}^{\text{del}} = y_i^{\text{sta}} + 0.3 \sin \left(\alpha_{i,k}^{\text{lim}} \times \frac{\pi}{180} \right) \quad (7)$$

Similarly, directivity lines of antennas are calculated for the visualization. For the antenna j of the base station i , we calculate the latitude $x_{i,j}^{\text{dir}}$ and the longitude $y_{i,j}^{\text{dir}}$ of the extremity of the antenna directivity line as follows:

$$x_{i,j}^{\text{dir}} = x_i^{\text{sta}} + 0.0005 \cos \left(\alpha_{i,j} \times \frac{\pi}{180} \right) \quad (8)$$

$$y_{i,j}^{\text{dir}} = y_i^{\text{sta}} + 0.0005 \sin \left(\alpha_{i,j} \times \frac{\pi}{180} \right) \quad (9)$$

Cartoradio processing output files

After these calculations done, antennas data are produced in CSV files which correspond to their operator. As said before, each operators have two CSV files: an output "sites" file, which contains data of kept antennas, and a "rejected" file which contains data of rejected antennas (underground antennas). These file contains the following entries:

- **BS_ANTENNA**: contains antenna information, such as base station and antenna numbers, latitude and longitude of the base station, height, postal address, azimuth, directivity and sector information...

- **DELIMITER**: represents a delimiter which separates two sectors, contains data about antenna associated to the delimiter (base station and antenna numbers), latitude and longitude of the base station, and latitude and longitude of the extremity of the delimiter.

Base station and measurements association process

The next functionality has for purpose to link measurements to a base station using EARFCN and PCI. During a measurement campaign on an urban-size area, we can considerate there is great chances that every EARFCN / PCI couple associated to measurements is unique, and we can use these couples to find to which base station a measurement point is related.

There is no trivial link between antennas and EARFCN / PCI, and it is necessary to calculate mathematical indicators. We will re-use criteria calculations used in the previous version of the program, and therefore base this part and the next formulas over [1] work: however, we will attempt reformulate more clearly and more exactly used calculations.

Input data of the assoiation process

First, this calculation takes two files as input: the file produced as output of the first step of AOF file data extraction (step 1 file), and the file produced as output of the second step of Cartoradio files data extraction (step 2 file) ; this calculation produces as output an association file.

The calculation is done with the class `association.CellAssociator`. The step 1 file is first read with the private function `CellAssociator._read_measurements`. `CELLINFO` and `SERVING_CELL` entries are stored in memory in DataFrame like structures, and `MEASUREMENTS` entries are directly written in the output file; `MEAS_EARFCNS` and `MEAS_PCIS` are also written in the output file, but are also used to create a list of EARFCN / PCI for further treatments. Then, the step 2 file is read with the private function `CellAssociator._read_measurements`: data from `BS_ANTENNA` entries are loaded into memory, and `DELIMITERS` entries are just copied in the output file. Data in memory are stored in two DataFrame, one which stores antennas and the other which stores associates `CELLINFO` data and serving cell measurements.

Geometric calculation on cells

Using these data, we calculate first a theoretical estimation of the cells, using Voronoi diagrams, from the positions of the sites. This calculation, using here the class `scipy.spatial.Voronoi` from the SciPy library, creates geometrical cells around each site. The cell of a site corresponds to the set of point which are nearest of this base station than other base stations.

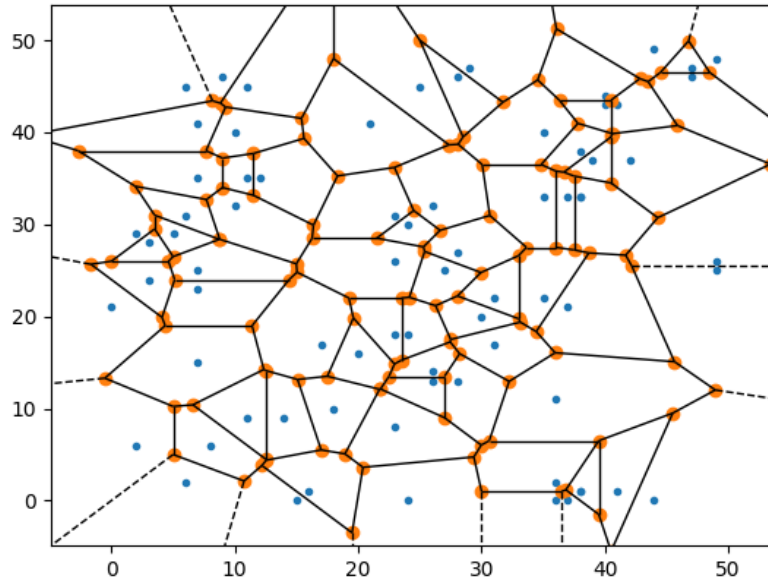


FIGURE 18 – Voronoi diagram of a set of 75 points on $\llbracket 0, 49 \rrbracket^2$ randomly generated with `random.randrange`, using SciPy's `Voronoi` class, `scipy.spatial.voronoi_plot_2d`, and `matplotlib.pyplot`.

After the initial cell calculations, it is required to remove "open" cells: these cells are situated at the extremities of the set of points, and extend in infinite directions. In SciPy, input points (base station points here) are stored in the array `Voronoi.points`. For each point of index `i`, corresponding region number is stored in the array `Voronoi.point_region`, at the index `i`. Then, we can finally get the corresponding region with `Voronoi.regions` array, each element of this array being a list of indexes of elements from `Voronoi.vertices` [6]. In the case of the current cell is an open cell, at least one vertex will be represented by the index `-1` under default parameters. [6]. We can eliminate these cells, as the calculation requires cells are convex polygons. Results of this calculation are stored in a DataFrame-like dictionary containing latitude, longitude of each site and the corresponding region number.

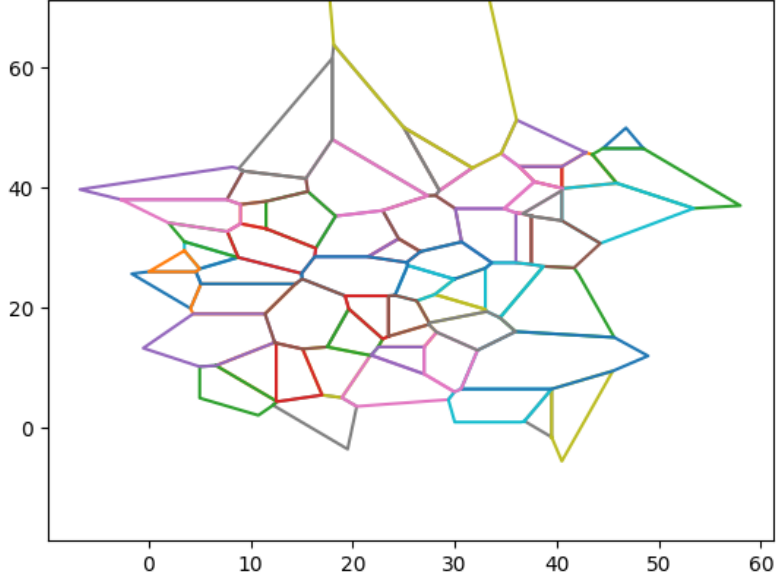


FIGURE 19 – Voronoi diagram of the same set of points than the last figure, after removing open cells, represented using `shapely` and `matplotlib.pyplot`.

We merge then antennas and Voronoi DataFrame issued from Voronoi dictionary over the latitude and longitude, to associate to each antenna the corresponding Voronoi cell. Following [1] notations, we will now name $\mathcal{V}_i$ the Voronoi cell of the base station i .

Then, for each group of serving cell measured points of the point association DataFrame with the same EARFCN / PCI, we will calculate the convex hull of these points, using the Shapely library and its class `shapely.MultiPoint`, and the property `convex_hull` of this class. Mathematically, the convex hull of set of points is the smallest convex polygon that contains these point, i.e. the smallest possible polygon which contains all point of the set, and where for two arbitrary points inside the polygon, the line between the two points is also in the polygon. Also according [1] notation, for a group of points of same EARFCN / PCI that we will note now $\mathcal{S}_n$, $n \in \mathbb{N}$, we will now use $\mathcal{C}_n$ to designate the corresponding convex hull of this group.

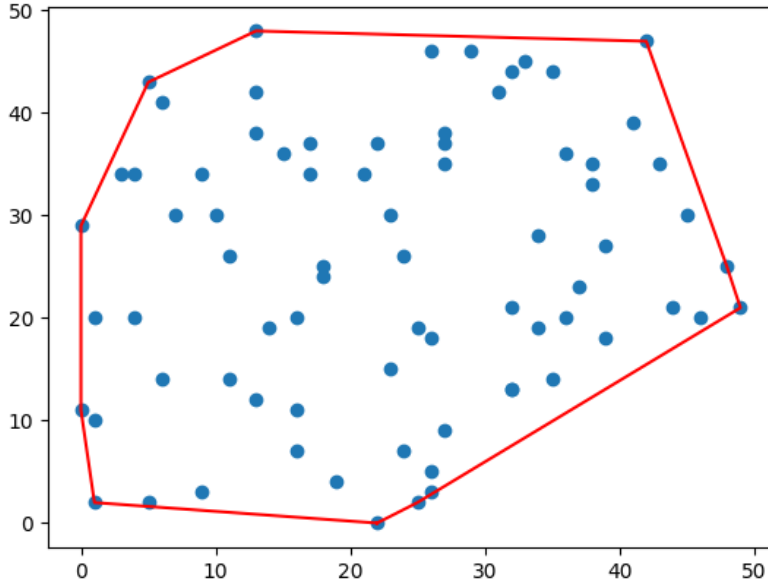


FIGURE 20 – Set of 75 points on $\llbracket 0, 49 \rrbracket^2$ randomly generated with `random.randrange` and its convex hull in red, using `convex_hull` property of `shapely.geometry.MultiPoint`, plotted with `matplotlib.pyplot`.

Use of vectorized function for the calculation

For the following part, we will first introduce the concept of vectorized functions, massively used in this part through the usage of the NumPy library.

Let $f : E \rightarrow F$ an arbitrary function mapping elements from E to F . Note there is no particular constraint on the function unless we can considerate f as a total function in the case of the Python programming language, as a function with without return value will return always `None`. Ordinarily, to calculate the value of F associated to $e \in E$ with f , we need just to calculate $f(e)$. That implies if we want to calculate a sequence $v : \llbracket 0, L \rrbracket \rightarrow F$, $L \in \mathbb{N}$ of numbers associated to the corresponding sequence $u : \llbracket 0, L \rrbracket \rightarrow E$ of numbers from E , we need to iterate over the sequence u following $k \in \llbracket 0, L \rrbracket$ to calculate $v_k = f(u_k)$ using for example in an imperative programming language a `for` loop, which can be really redundant in program.

A solution to this is the use of vectorized functions. Instead of iterating over sequence of numbers from E to calculate the corresponding sequence of numbers of F , taking the example of u and v , we can use vectors $\vec{u}$ and $\vec{v}$, with:

$$\vec{u} = \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_L \end{bmatrix} \quad \text{and} \quad \vec{v} = \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ \vdots \\ v_L \end{bmatrix} \quad (10)$$

A vectorized function follows this principle: instead of mapping an element of a arbitrary set E to another element of another set of F using a function f , it is possible to map a series of $n \in \mathbb{N}$ elements from E represented with a vector from E^n to a vector from F^n , representing the

associated series of number from F , using the vectorized version f_V of the function f , with the following definition, with $e_k \in E$ and $f_k \in K$, $k \in \llbracket 0, n \rrbracket$:

$$f_V : E^n \longrightarrow F^n \quad (11)$$

$$\begin{bmatrix} e_0 \\ e_1 \\ e_2 \\ \vdots \\ e_n \end{bmatrix} \longmapsto \begin{bmatrix} f(e_0) \\ f(e_1) \\ f(e_2) \\ \vdots \\ f(e_n) \end{bmatrix} \quad (12)$$

Using NumPy, it is possible to use as vector lists or NumPy arrays. A lot of standard mathematical functions have a vectorized version in NumPy (`numpy.cos`, `numpy.abs`...), and it is possible to create the vectorized version of an arbitrary function using `numpy.vectorize`.

The reason to use these vectorized functions is they allow to calculate values over arrays of number concisely. We will find this situation in the following calculations.

Cell association calculation

In order to associate each measurement to an antenna, [1] uses several criteria. These criteria are based on the angle between the north axis and the line between the point and the corresponding site, and on the belonging of the point to a cell.

To do properly angular-based calculation, latitude and longitude, expressed in degrees, which are spheric coordinates, are projected over a plane: calculation used in [1] to do so are the following: let $A_i = (\phi_i, \lambda_i)$ geolocation of the i -th site, and $X_k = (\phi_k, \lambda_k)$ geolocation of a measurement, with $i, k \in \mathbb{N}$. We calculate relative coordinates $(d_{N,i,k}, d_{E,i,k})$ over plane projection of the point X_k , following the site A_i . These relative coordinates are calculated in the old program, based on [1] as the following:

$$d_{N,i,k} = \frac{\pi}{180} (\phi_k - \phi_i) \quad (13)$$

$$d_{E,i,k} = \frac{\pi}{180} (\lambda_k - \lambda_i) \cos(\phi_i) \quad (14)$$

`numpy.array` containing latitudes and longitudes are used for this calculus, as they use vectorized mathematical operators overloads; `numpy.cos` function is also used.

From these coordinates, we calculate the angle $\alpha(A_i, X_k)$, which is given by the following formula, defined in [1] :

$$\alpha(A_i, X_k) = \arctan 2(d_{E,i,k}, d_{N,i,k}) \quad (15)$$

`numpy.arctan2` vectorized function is used here.

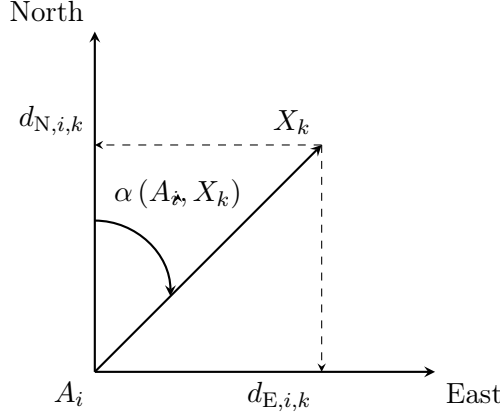


FIGURE 21 – Relative coordinates of X_k from A_i and angle $\alpha(A_i, X_k)$. Figure by Hoang TRIEU, extracted from [1].

Then, it is possible to calculate association criteria. First, we attribute to each point of indice $k \in \mathbb{N}$ in a group $\mathcal{S}_n$, $n \in \mathbb{N}$ with a RSRP R_k a weight which is given by the following formula [1]:

$$w_k = \begin{cases} 0.15 & \text{if } R_k \leq -100 \text{ dBm} \\ 0.6 & \text{if } -100 \text{ dBm} < R_k < -80 \text{ dBm} \\ 1 & \text{if } R_k \geq -80 \text{ dBm} \end{cases} \quad (16)$$

Weight function is vectorized in the python program using `numpy.vectorize`.

Based on these weights, we calculate now for the group $\mathcal{C}_n$ the angular criteria $\sigma_{i,j,n}$ for each antenna of indice $j \in \mathbb{N}$ from the site of indice $i \in \mathbb{N}$ [1]:

$$\sigma_{i,j,n} = \begin{cases} 0 & \text{if } \mathcal{V}_i \cap \mathcal{C}_n = \emptyset \\ \frac{\sum_{k=1}^{|\mathcal{C}_n|} w_k \frac{[\alpha_{i,j}^{\min} < x < \alpha_{i,j}^{\max}](\alpha(A_i, X_k))}{|S_k|}}{|\mathcal{C}_n|} & \text{otherwise} \end{cases} \quad (17)$$

This indicator is calculated using a function vectorized with `numpy.vectorize`.

Note that we use the notation $[\alpha_{i,j}^{\min} < x < \alpha_{i,j}^{\max}](\alpha(A_i, X_k))$. This notation is know as Iverson bracket. Let P be a first-order logic predicate over a domain E . We define the Iverson bracket $[P]$ as follows ($\neg$ means not):

$$[P] : E \longrightarrow \{0, 1\} \quad (18)$$

$$x \longmapsto \begin{cases} 0 & \text{if } \neg P(x) \\ 1 & \text{if } P(x) \end{cases} \quad (19)$$

This mean that if we pass as argument of $[P]$ a value x , if x follows the condition defined by P , the function will return 1, if not it will return 0. Here, we use as condition that the angle $\alpha(A_i, X_k)$ must be included between the minimum and maximum sector of the antenna j of the site i . This criteria consists of the average of the belonging value of points of $\mathcal{C}_k$ to the sector of a given antenna.

The second criteria is the cell belonging criteria of the points of $\mathcal{C}_n$ to the cell $\mathcal{V}_i$, with taking account of the belonging of the point to the sector of the antenna j . This indicator is noted $\psi_{i,j,n}$ [1]:

$$\psi_{i,j,n} = \begin{cases} 0 & \text{if } \mathcal{V}_i \cap \mathcal{C}_n = \emptyset \\ \sum_{k=1}^{|\mathcal{C}_n|} w_k \frac{\mathbb{1}_{\mathcal{V}_i}(X_k) \times [\alpha_{i,j}^{\min} < x < \alpha_{i,j}^{\max}](\alpha(A_i, X_k))}{|S_k|} & \text{otherwise} \end{cases} \quad (20)$$

The notation $\mathbb{1}_{\mathcal{V}_i}$ corresponds to an indicator function, which is a particular case of Iverson brackets defined as follows: $\mathbb{1}_A = [x \in A]$, A being an arbitrary set. In other words, the function will return 1 if a given argument x belongs to A , 0 if not. Here, we want to check if the point belongs to the Voronoi cell of the antenna.

The indicator is calculated following using by multiplying the result vector of calculations of $\mathbb{1}_{\mathcal{V}_i}(X_k)$ by the result vector of calculations of $w_k \times [\alpha_{i,j}^{\min} < x < \alpha_{i,j}^{\max}](\alpha(A_i, X_k))$, kept from the calculation of the first criteria, then calculate the average of the obtained vector.

With $\sigma_{i,j,n}$ and $\psi_{i,j,n}$ being calculated, these indicators need to be merged into a final indicator, called $\theta_{i,j,n}$, defined as follows [1]:

$$\theta_{i,j,n} = \begin{cases} 0 & \text{if } \psi_{i,j,n} < 0.25 \\ 0.2\psi_{i,j,n} + 0.8\sigma_{i,j,n} & \text{otherwise.} \end{cases} \quad (21)$$

The calculation of $\theta_{i,j,n}$ is not vectorized, as its calculation is based on only two values, $\sigma_{i,j,n}$ and $\psi_{i,j,n}$, and therefore does not need to be calculated from a large series of values.

The presence of the threshold value 0.25 allows to exclude group of points very partially included in a sector, which can be considered as not significant for the association [1].

θ criteria is calculated for all antennas j from all cells i , for each group n . Then, we pose the value $\theta_{i_{\max},j_{\max},n}$ as the maximum θ value associated to the group of point S_n . Association of S_n to the antenna j of the site i will be done only if [1]:

$$\forall i', j', \left(i_{\max} \neq i' \wedge j_{\max} \neq j' \wedge \frac{\theta_{i',j',n}}{\theta_{i_{\max},j_{\max},n}} \neq 0 \wedge \theta_{i_{\max},j_{\max},n} \neq 0 \right) \quad (22)$$

$$\implies \left| \log \left(\frac{\theta_{i',j',n}}{\theta_{i_{\max},j_{\max},n}} \right) \right| > 0.08 \quad (23)$$

i. e. for all i' and j' , the condition in (22) is false or the condition at the left of the arrow in (23) is true.

Using this condition, several antennas can be associated to a same group, as surface in the angle of the sector of an antenna extends infinitely. The last step of the treatment consists to choose the group $\mathcal{S}_n$ associated to the convex polygon $\mathcal{C}_n$ such as the distance between the associate antenna and the nearest point of $\mathcal{C}_n$ is minimal [1]. This calculation is done using Shapely library, by calculating a projection number on the polygon, which can be used to interpolate the corresponding point over the polygon.

Warning

There can be several groups of measurement point for the same sector. Only one should be kept but this is not implemented.

Content of the association output file

The output file⁷ contains the following fields:

- **MEAS_EARFCNS**: list of all EARFCNs in (EARFCN, PCI) pairs found during the measurement campaign.
- **MEAS_PCIS**: list of all PCIs in (EARFCN, PCI) pairs found during the measurement campaign.
- **MEASUREMENT**: global measurements associated to (EARFCN, PCI) pairs following columns. Can be RSRP, RSRQ or RSSI measurements.
- **DELIMITER**: used to draw antennas sectors delimitation lines.
- **BS_ANT_DIR**: used to draw antennas directivity lines.
- **ASSOC**: defines an association between a Cartoradio number (base station), an antenna number and TAC, CID, EARFCN, PCI values.
- **POINT**: defines a point with its latitude and longitude, its TAC, CID, EARFCN and PCI values and its measurements (RSRP, RSRQ, RSSI, CINR).

⁷If the input file is xxx then the output file is **assoc_xxx**

III. Visualization

This part aims to document the web front-end application used to visualize measurement campaigns. We will first focus on the general structure of this part, before presenting briefly the principles of working of this application.

JavaScript application structure

This application, stored in the folder `web_ui`, is splitted into several files, which are themselves stored in several folder.

In the main folder, the application web page is stored in `index.html`. This file is a Hyper-Text Markup Language File (HTML), and contains the layout of the application. Several styles are applied on this web page, user defined styles being stored in the `styles` folder, in the file `frontstyle.css`, which is a Cascading Style Sheet (CSS) file.

However, the main application uses a JavaScript program stored in the folder `scripts`, which contains the following JavaScript files:

- `app.js`: main app file. It contains the class `app.App`, which wraps the global state of the application, such as the map or selected data, defines events handlers that allows the user to interact with the application, and updates the user interface when necessary.
- `csvread.js`: this file contains the class `csvread.CSVReader`, which is used to read the CSV file passed as an input of the application. Data from the file are also stored in this class.
- `drawing.js`: this file contains the class `drawing.Map`, which can wrap the map component of the user interface, and is used to draw layers on the map, following the user selection, with dedicated methods.
- `processing.js`: processes antennas, cells and sectors data into drawable geographical informations.
- `styles.js`: this file defines function to style components of the map (points, cells, heatmap...).
- `utils.js`: this file contains functions for various utilities, which are not classable in the other files (array manipulation, object copy...).

The program also use several libraries, stored in the `library` folder⁸ which are the following:

- Bootstrap: used to define the layout of the user interface.
- Leaflet: defines the map component `L.Map` used in the UI.
- d3: *Data Driven Documents*, used as a dependency to visualize data.
- Leaflet-d3: extension to Leaflet and d3 to visualize data, used for heatmaps.
- turf.js: used to do calculations over the GeoJSON format⁹

⁸Note there in this folder several legacy unused libraries.

⁹Format of JavaScript objects standardized by the RFC-7946 (<https://www.rfc-editor.org/rfc/rfc7946.txt>)

Aspects of the inner working of the application

Selection of EARFCNs and PCIs

One of the most important features of the application is the selection by the use of EARFCNs and PCIs of data to be displayed. This selection is mostly managed by the `utils.subEarpcis` function, but is also managed by other functions. Usually, these functions takes two parameters, `earfcns` and `pcis`, which have as default values the value `null`. An `earfcns` parameter which is `null` means that every EARFCN need to be selected, a `pcis` parameter which is `null` means that every PCI need to be selected, and two `null` parameters means that every EARFCN / PCI must be selected. To select no EARFCN / PCI pair, the two parameters must be empty arrays (`[]`). To select existing EARFCNs and PCIs, the `earfcns` parameter must contains EARFCNs of choosen pairs, and `pcis` parameter must contain corresponding PCIs.

Management of layers

Following selected EARFCNs and PCIs, layers of data to display on the map must be managed. The only option to hide a layer using Leaflet is to remove it from the map; that implies layers must be stored in memory, and must be identifiable.

Measurement layers, using hexagonal-tiled heatmaps, are modified by removing their points of data, and filtering then re-adding them following selected EARFCNs and PCIs. These data points are stored in arrays and contains a longitude, a latitude and value. Serving measurements layers and global measurement layers are displayed separately, following if a the user selected serving measurements or not.

Point layers (TAC and PCI) consist in fact in groups of layers. These layers are stored in dictionary-like objects, each associated to an EARFCN and PCI, and are added or removed from the following the user selection.

Antennas and theoretical cell layers are managed more easily, as they are GeoJSON object created with `turf.js`: in particular, the cell layer contains the Voronoi diagram of theoretical cells, generated by `turf.js` from antenna datas.

Associated site layer consists in a group of tooltips markers, generated from antennas and association data, and bound to a tooltip which contains checkboxes for the selection of EARFCNs and PCIs by sites. These checkboxes are added from the current user selection through EARFCN and PCI selection drop down menus, and are automatically checked or not following the previous checkbox selection before the drawing of the mark.

Automatic generation of the color gradient for heatmaps

Heatmap gradient use cold colors for low values, "intermediate" colors for intermediate values, and hot colors for high values. This gradient is calculated by Leaflet-d3, using a list of given color values.

This list can be generated automatically, using the HSL (Hue-Saturation-Lightness) color system. This color system defines each color over three values $(h, s, l) \in [0, 360] \times [0, 1] \times [0, 1]$. We can generate the required colors by decreasing h value over $[0, 240]$, with $s = 1$ to have fully saturated color and $l = 0.5$ to not have too "white" or too "black" colors. Following this principle, to generate a set of n , color, we can decrease h by $\frac{240}{n}$ steps. We can use to do easily this the d3 library, with `d3.hsl` function, and use on the return value of this function the function `formatHex()` to obtain the hexadimal RGB value of the color.

Updating policy of the UI

As the user can frequently filter data on the visualization, several functions are defined in order to update the user interface. They are all situated in the file `app.js`:

- `app.update`: updates layers using selected EARFCN and PCIs by redrawing them. Called on every change of selected EARFCN and / or PCI, from selectors or checkboxes.
- `app.updateDisplay`: hide or show layer, which is different from updating their data. Called every time

Bibliography

- [1] H. Trieu, “Visualization and characterization of the coverage of commercial 4G base stations,” en, IMT Atlantique, Rennes, Tech. Rep., 2019, p. 23.
- [2] Y. Bouguen, E. Hardouin, F.-X. Wolff, G. Pujolle, and A. Maloberti, *LTE et les réseaux 4G*, fr-FR. Paris: Eyrolles, 2012, ISBN: 978-2-212-12990-8.
- [3] K. Callac, “Rapport de stage de L3 Informatique,” fr-FR, IMT Atlantique, Rennes, Tech. Rep., 2022.
- [4] X. Lagrange, *Xavier LAGRANGE / IMT Atlantique - FAQ Technique*, fr-FR. [Online]. Available: <https://www.imt-atlantique.fr/fr/personne/xavier-lagrange#edit-group-cours-enseignements> (visited on 06/20/2022).
- [5] F. Launay, *RSRP et RSRQ 2ème : Définition*, fr-FR, Aug. 2013. [Online]. Available: <https://blogs.univ-poitiers.fr/f-launay/2013/08/27/rsrp-et-rsrq-2eme-definition/> (visited on 07/11/2022).
- [6] The SciPy Community, *Scipy.spatial.Voronoi — SciPy v1.8.1 Manual*, en. [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.Voronoi.html> (visited on 07/25/2022).